

Score Matching

Andrés C. Medina

1. Desarrollo teórico

(a) **Camino de probabilidad Gaussiano y score condicional.** Elegimos el camino *CondOT*: $\alpha_t = t$, $\beta_t = 1 - t$, $t \in [0, 1]$, que cumple $\alpha_0 = \beta_1 = 0$, $\alpha_1 = \beta_0 = 1$ (en $t = 0$ el camino es ruido puro $\mathcal{N}(0, I_d)$; en $t = 1$, el dato limpio δ_z):

$$p_t(\cdot | z) = \mathcal{N}(tz, (1-t)^2 I_d), \quad \nabla \log p_t(x | z) = -\frac{x - tz}{(1-t)^2}.$$

Se prefirió por su simplicidad: α_t, β_t lineales en t dan fórmulas cerradas simples para el campo de velocidad y el score condicionales (ítem (c)), evitando *schedulers* más complejos (p.ej. coseno o el de DDPM).

(b) **Función de pérdida $\mathcal{L}$ y su relación con $\mathcal{L}_{\text{SM}}$.** La pérdida de score matching marginal $\mathcal{L}_{\text{SM}}(\theta) = \frac{1}{2} \mathbb{E}_{p_t} \|s_\theta(x, t) - \nabla \log p_t(x)\|^2$ es intratable pues $\nabla \log p_t(x)$ es desconocido. El truco de marginalización (Teorema 3.3 y su extensión a DSM, Teorema 4.4, en [score_matching_teoría.pdf](#); Teorema 20 en el apunte MIT) prueba que $\mathcal{L}_{\text{SM}}(\theta) = \mathcal{L}_{\text{CSM}}(\theta) + C$ con C independiente de θ , donde

$$\mathcal{L}_{\text{CSM}}(\theta) = \mathbb{E}_{t, z \sim p_{\text{data}}, x \sim p_t(\cdot | z)} \|s_\theta(x, t) - \nabla \log p_t(x | z)\|^2$$

si es tratable, ya que $\nabla \log p_t(x | z)$ es conocido en forma cerrada (ítem (a)). Reparametrizando $s_\theta = -\epsilon_\theta / \beta_t$ y sustituyendo $x = \alpha_t z + \beta_t \varepsilon$ se obtiene exactamente la pérdida que optimiza `train.py`:

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \text{Unif}[0,1], z \sim p_{\text{data}}, \varepsilon \sim \mathcal{N}(0, I)} \|\epsilon_\theta(\alpha_t z + \beta_t \varepsilon, t) - \varepsilon\|^2.$$

Como $\mathcal{L}$ y $\mathcal{L}_{\text{CSM}}$ difieren solo por el cambio de variable $s_\theta \leftrightarrow \epsilon_\theta$ (biyección que no depende de θ), tienen el mismo minimizador; luego minimizar $\mathcal{L}$ equivale a minimizar $\mathcal{L}_{\text{SM}}$.

(c) **Relación entre s_θ , ϵ_θ y u_θ .** Nuestra red parametriza ϵ_θ (predicador de ruido), no s_θ ni u_θ directamente: evita dividir por β_t durante el entrenamiento (inestable cuando $\beta_t \rightarrow 0$, i.e. $t \rightarrow 1$) y es la parametrización estándar de DDPM. Para caminos Gaussianos, la Proposición 1 del apunte MIT da la conversión exacta

$$u_t(x | z) = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \nabla \log p_t(x | z) + \frac{\dot{\alpha}_t}{\alpha_t} x,$$

que para CondOT ($\dot{\alpha}_t = 1$, $\dot{\beta}_t = -1$) se reduce a $u_\theta(x, t) = (x - \epsilon_\theta(x, t))/t$; análogamente $s_\theta(x, t) = -\epsilon_\theta(x, t)/\beta_t$. En consecuencia basta entrenar una sola parametrización (ϵ_θ): las otras dos se recuperan algebraicamente, sin reentrenar, y ambas se usan en el muestreo (ítem (e)).

(d) **Regla de *classifier-free guidance*.** Por Bayes, $\nabla \log p_t(x | y) = \nabla \log p_t(x) + \nabla \log p_t(y | x)$; amplificar artificialmente el término “clasificador” $\nabla \log p_t(y | x)$ por una escala w da la regla de CFG

$$\tilde{s}_t(x | y) = (1 - w) \nabla \log p_t(x | \emptyset) + w \nabla \log p_t(x | y).$$

Para $w = 1$ se recupera exactamente el score guiado verdadero $\nabla \log p_t(x | y)$; para $w \neq 1$ es una **heurística**: no es el score de ninguna densidad real, solo refuerza artificialmente la señal de la etiqueta. Entrenamos dos redes con la misma arquitectura (`model.py`), que difieren solo en su entrenamiento:

Red condicional pura (`modelo_cond.pt`, `label_dropout=0`, sin token nulo): y real siempre, pérdida DSM estándar del ítem (b); se muestrea con $w = 1$, sin necesitar $\emptyset$.

Red con CFG (`modelo_cfg.pt`, `label_dropout=η=0.1`, con token nulo $\emptyset$): y se reemplaza por $\emptyset$ con probabilidad η , independiente por muestra (entrenamiento CFG, apunte MIT):

$$\mathcal{L}_{\text{CFG}}(\theta) = \mathbb{E}_{t, z, \varepsilon, \tilde{y}} \|\epsilon_\theta(\alpha_t z + \beta_t \varepsilon, t, \tilde{y}) - \varepsilon\|^2, \quad \tilde{y} = \begin{cases} \emptyset & \text{con prob. } \eta \\ y & \text{en otro caso.} \end{cases}$$

En inferencia se combina $\tilde{\varepsilon} = (1 - w) \epsilon_\theta(x, t, \emptyset) + w \epsilon_\theta(x, t, y)$ —equivalente a combinar s_θ o u_θ por separado, al ser ambas transformaciones afines de ϵ_θ (ítem (c))—, con el w elegido y justificado en Resultados.

(e) **Algoritmo de muestreo.** Integramos la SDE reversa vía Euler–Maruyama aplicando la regla de CFG del ítem (d) (Algoritmo 1); $T_{\min} > 0$ evita dividir por $\alpha_t \approx 0$ cerca de $t = 0$.

Algorithm 1 Muestreo condicional con CFG (Euler–Maruyama)

Require: ϵ_θ entrenada, etiqueta y , escala w , pasos n , σ_{coef} , $T_{\min}$

```

1:  $h \leftarrow 1/n$ ;  $t \leftarrow 0$ ;  $X \sim \mathcal{N}(0, I_d)$ 
2: for  $i = 1, \dots, n$  do
3:    $t' \leftarrow \max(t, T_{\min})$ 
4:   if  $w = 1$  then
5:      $\tilde{\epsilon} \leftarrow \epsilon_\theta(X, t', y)$ 
6:   else
7:      $\tilde{\epsilon} \leftarrow (1 - w) \epsilon_\theta(X, t', \emptyset) + w \epsilon_\theta(X, t', y)$ 
8:   end if
9:    $u \leftarrow (X - \tilde{\epsilon})/t'$ ;  $s \leftarrow -\tilde{\epsilon}/(1 - t')$ ;  $\sigma \leftarrow \sigma_{\text{coef}}(1 - t')$ 
10:   $z \sim \mathcal{N}(0, I_d)$ 
11:   $X \leftarrow X + h[u + \frac{\sigma^2}{2}s] + \sqrt{h}\sigma z$ 
12:   $t \leftarrow t + h$ 
13: end for
14: return  $X$ 

```

2. Resultados

Arquitectura. *Difusión* (`model.py`): UNet 2D condicional, canales $1 \rightarrow 32 \rightarrow 64 \rightarrow 128$ con *downsampling* $28 \rightarrow 14 \rightarrow 7$ y decoder simétrico con *skip connections*; t (embedding sinusoidal) e y (embedding con token nulo $\emptyset$) se inyectan de forma aditiva (FiLM) en cada bloque residual; $\sim 1.27\text{M}$ parámetros (detalle por capa en el Apéndice A). *Clasificador* (`model_clf.py`): CNN de 3 bloques `conv+MaxPool` + MLP, $\sim 1.70\text{M}$ parámetros. Optimizador Adam, $\text{lr} = 3 \times 10^{-4}$ (difusión) / 10^{-3} (clasificador), `batch_size` = 256, 40 épocas (redes de difusión) / 8 épocas (clasificador), `label_dropout` = 0.1 para `modelo_cfg`, semilla fija (`seed=0`), entrenado en GPU (Colab, T4).

Validación experimental. Con el clasificador como juez automático (1000 muestras/red, 1000 pasos de Euler–Maruyama, $w = 3$ para `modelo_cfg` — justificación de w más abajo):

Red	Fidelidad promedio	Diversidad promedio
<code>modelo_cond</code> ($w = 1$)	93.1 % (rango 89–97 % por clase)	17.27
<code>modelo_cfg</code> ($w = 3$)	100 % (las 10 clases)	16.65

`modelo_cfg` alcanza fidelidad perfecta a cambio de solo $\sim 3.6\%$ menos diversidad intra-clase (distancia media por pares en espacio de píxeles) — el compromiso esperado de la naturaleza heurística de CFG (ítem (d)): al amplificar artificialmente $\nabla \log p_t(y | x)$, las muestras se concentran más cerca de los modos de alta verosimilitud bajo el clasificador, sacrificando algo de variabilidad. Elegimos $w = 3$ por ser el punto donde la fidelidad ya satura en 100 % sin una caída mayor de diversidad (Fig. 1).

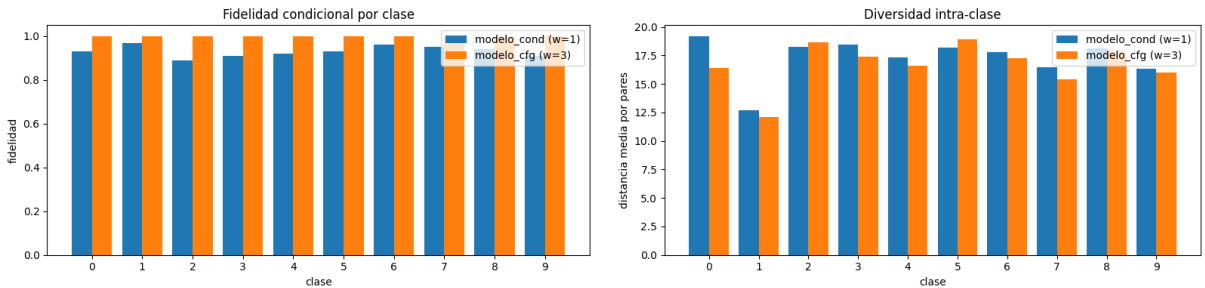


Figura 1: Fidelidad y diversidad por clase, `modelo_cond` ($w = 1$) vs. `modelo_cfg` ($w = 3$).

Curvas de pérdida. Ambas redes de difusión convergen de forma limpia y estable de ~ 1.1 a ~ 0.04 – 0.05 (MSE de ruido) en 40 épocas (Fig. 2), sin señales de inestabilidad ni sobreajuste.

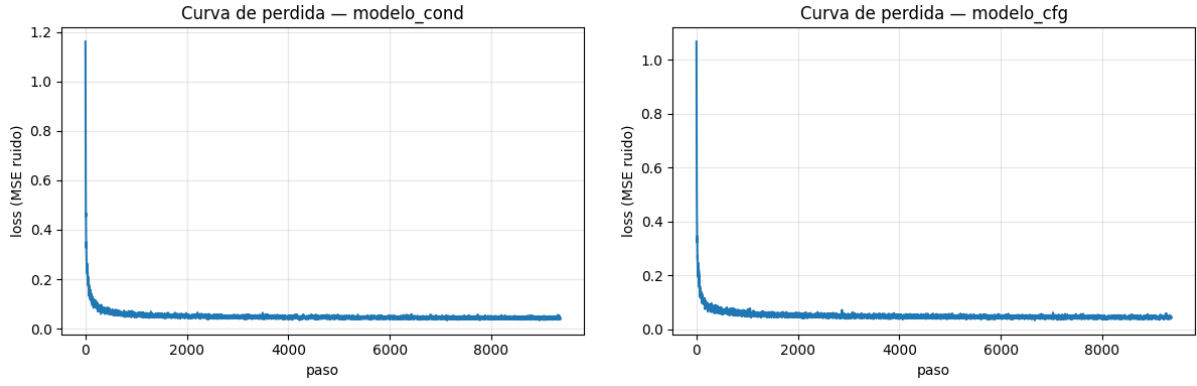


Figura 2: Curva de pérdida de entrenamiento: `modelo_cond` (izq.) y `modelo_cfg` (der.).

Ejemplos de generación. La Fig. 3 compara las tres configuraciones evaluadas (una fila por clase, 0–9): `modelo_cond` con $w = 1$, `modelo_cfg` con $w = 1$ (sin guía) y `modelo_cfg` con $w = 3$ (guía elegida). Los dígitos son claramente reconocibles en las tres, pero $w = 3$ produce trazos visiblemente más nítidos y uniformes —consistente con la fidelidad del 100 % reportada arriba— a cambio de menos variación visual entre las tres muestras de cada fila.

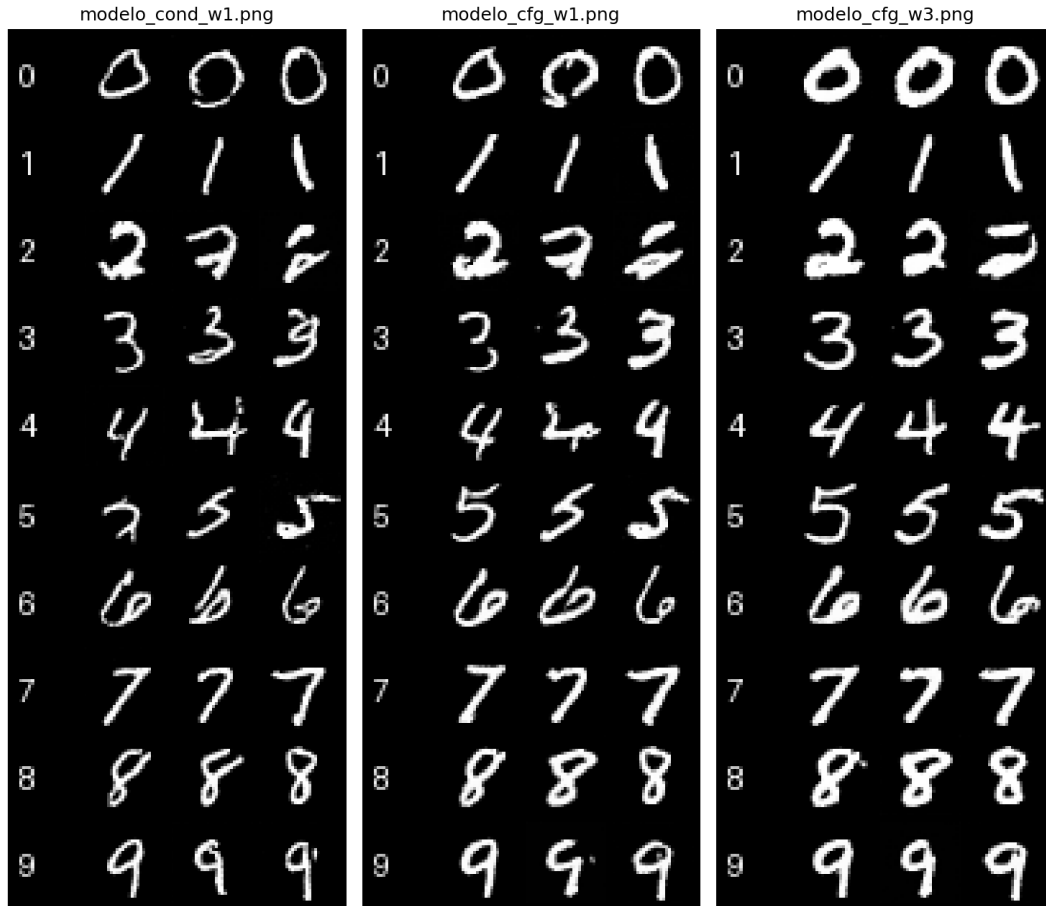


Figura 3: Muestras generadas, una fila por clase (0–9). Izquierda: `modelo_cond` ($w = 1$). Centro: `modelo_cfg` sin guía ($w = 1$). Derecha: `modelo_cfg` con la guía elegida ($w = 3$).

Referencias

1. A. Medina. *Score Matching: Fundamentos Matemáticos, Demostraciones Completas y Aplicación al Análisis de Imágenes*. Documento de estudio, repositorio `score_matching`, 2026.
2. P. Holderrieth, E. Erives. *An Introduction to Flow Matching and Diffusion Models*. MIT Class 6.S184, 2025.

3. J. Ho, A. Jain, P. Abbeel. *Denoising Diffusion Probabilistic Models*. NeurIPS, 2020.
4. J. Ho, T. Salimans. *Classifier-Free Diffusion Guidance*. arXiv:2207.12598, 2022.

A. Cálculos

Diagramas de arquitectura. Las Figuras 4 y 5 muestran los diagramas completos de ambas redes, elaborados a mano verificando cada capa contra el código (`model.py`, `model_clf.py`) y sus checkpoints.

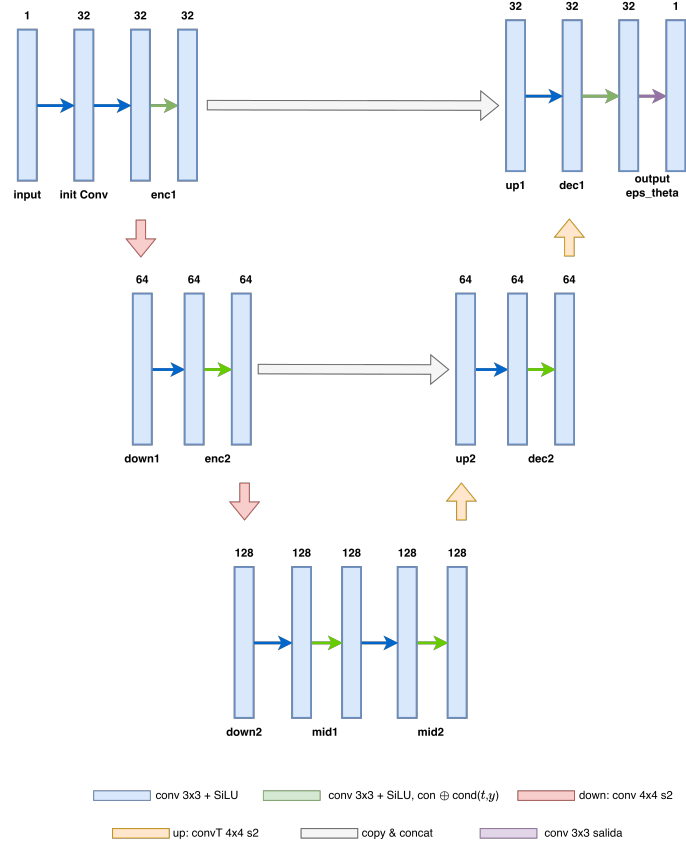


Figura 4: Arquitectura de la UNet condicional (`model.py`), en el formato clásico de Ronneberger et al. (2015). Cada rectángulo angosto es un mapa de características (el número arriba indica su cantidad de canales); flechas **azules**: `conv3×3+SiLU`; flechas **verdes**: `conv3×3+SiLU` con inyección de condicionamiento $\oplus = \text{cond}(t, y)$ (FiLM aditivo, ítem (d)) dentro de cada bloque residual (`enc1`, `enc2`, `mid1`, `mid2`, `dec2`, `dec1`); flechas **rojas/naranjas**: downsampling/upsampling *aprendido* (`conv4×4` stride 2 y su transpuesta, `down1/2` y `up1/2`); flechas **grises**: conexión de salto (*copy & concat*); flecha **púrpura**: convolución de salida (`out_conv`, `conv3×3`, sin condicionamiento). A diferencia del paper original, aquí no hay recorte espacial —todas las convoluciones 3×3 usan `padding=1`— ni *max-pooling*: el downsampling/upsampling es enteramente aprendido (ítem (a) y Arquitectura en Resultados).

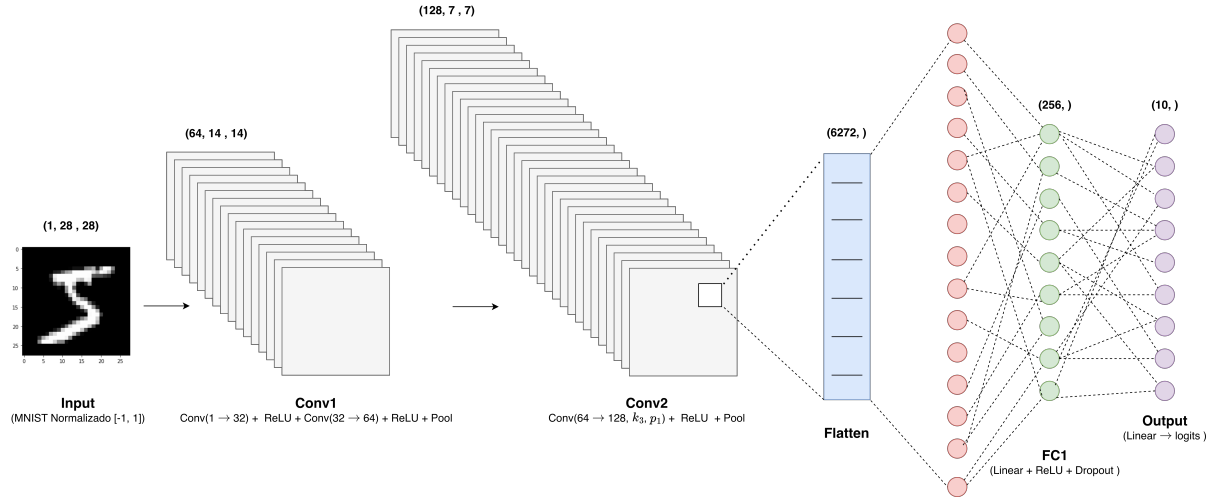


Figura 5: Arquitectura del clasificador auxiliar (`model_clf.py`), usado como juez automático en `evaluate.py`. Los prismas apilados representan mapas de características tras cada bloque convolucional (forma (`canales`, `alto`, `ancho`) indicada arriba); tras Conv Block 2 el tensor se aplanar (`Flatten`) en un vector de 6272 valores, sin parámetros propios —es solo un *reshape*—. Las *líneas punteadas* entre columnas de neuronas representan los pesos aprendidos de cada capa `Linear` (los parámetros viven en las conexiones, no en los círculos): rojo→verde es `FC1` (`Linear(6272→256)`, 1,605,888 parámetros) y verde→morado es la capa de salida (`Linear(256→10)`, 2,570 parámetros), que entrega los logits sobre los 10 dígitos.

Conteo de parámetros por bloque, UNet (`model.py`), total 1,270,817:

Bloque	Parámetros	Bloque	Parámetros
<code>time_mlp</code>	33,024	<code>mid1</code>	312,192
<code>label_embedding</code>	1,408	<code>mid2</code>	312,192
<code>cond_mlp</code>	16,512	<code>up2</code>	131,136
<code>init_conv</code>	320	<code>dec2</code>	127,616
<code>enc1</code>	22,752	<code>up1</code>	32,800
<code>down1</code>	32,832	<code>dec1</code>	34,112
<code>enc2</code>	82,368	<code>out_norm</code>	64
<code>down2</code>	131,200	<code>out_conv</code>	289

Conteo de parámetros por capa, CNN clasificador (`model_clf.py`), total 1,701,130:

Capa	Forma de salida	Parámetros
<code>Conv2d(1→32,k3,p1) + ReLU</code>	$32 \times 28 \times 28$	320
<code>Conv2d(32→64,k3,p1) + ReLU</code>	$64 \times 28 \times 28$	18,496
<code>MaxPool2d(2)</code>	$64 \times 14 \times 14$	0
<code>Conv2d(64→128,k3,p1) + ReLU</code>	$128 \times 14 \times 14$	73,856
<code>MaxPool2d(2)</code>	$128 \times 7 \times 7$	0
<code>Flatten</code>	6272	0
<code>Linear(6272→256) + ReLU</code>	256	1,605,888
<code>Dropout(0.5)</code>	256	0
<code>Linear(256→10)</code>	10	2,570

Suma: $320+18,496+73,856+1,605,888+2,570 = 1,701,130$, consistente con el total reportado por `sum(p.numel() for p in model.parameters())` sobre el checkpoint entregado.